

Relatório - Compressão de Arquivos-Texto com Algoritmo de Huffman

Aluno(s): Gabriel F. Obregon e Vinicius F. Munchen

Curso: Ciência da Computação

Disciplina: Projeto e Análise de Algoritmos

Professor: Romulo Cesar Silva

1. Objetivo

O objetivo deste trabalho é implementar um programa em Java para compactação e descompactação de arquivos-texto utilizando o algoritmo de Huffman. O sistema permite realizar a codificação por caractere e por palavra, conforme solicitado no enunciado do trabalho.

O programa apresenta um menu com as seguintes opções:

1. Compactar arquivo usando codificação por caractere;
2. Descompactar arquivo usando codificação por caractere;
3. Compactar arquivo usando codificação por palavra;
4. Descompactar arquivo usando codificação por palavra.

Em todas as opções, o usuário informa o arquivo de entrada e o arquivo de saída.

2. Visão geral do algoritmo de Huffman

O algoritmo de Huffman é um método de compressão sem perda. Isso significa que, após a compactação e a descompactação, o arquivo reconstruído deve ser idêntico ao arquivo original.

A ideia principal é atribuir códigos menores aos símbolos que aparecem com maior frequência e códigos maiores aos símbolos menos frequentes. Dessa forma, o texto passa a ser representado com uma quantidade menor de bits.

O processo geral utilizado no programa é:

1. Ler os símbolos do arquivo;
2. Contar a frequência de cada símbolo;
3. Construir a árvore de Huffman com base nas frequências;
4. Gerar o dicionário de códigos binários;
5. Substituir cada símbolo pelo respectivo código Huffman;
6. Gravar os bits compactados no arquivo de saída;
7. Gravar também o dicionário no arquivo compactado, permitindo a descompactação posterior.

3. Codificação por caractere

Na codificação por caractere, cada caractere do arquivo é tratado como um símbolo individual. Isso inclui letras, acentos, espaços, pontuações e quebras de linha.

Exemplo:

```
pão
```

Símbolos considerados:

```
p  
ã  
o
```

A contagem de frequência é feita para cada caractere. Depois disso, a árvore de Huffman é construída e cada caractere recebe um código binário de tamanho variável.

4. Codificação por palavra

Na codificação por palavra, o programa separa o texto em tokens. Um token pode ser uma sequência de letras ou uma sequência de não letras.

Exemplo:

```
Olá, mundo!
```

Tokens considerados:

```
"Olá"  
, "  
"mundo"  
!"
```

Essa decisão foi adotada para preservar espaços, pontuação e quebras de linha, permitindo que a descompactação reconstrua o arquivo exatamente igual ao original.

5. Estruturas de dados utilizadas

As principais estruturas de dados utilizadas foram:

- `HashMap<String, Long>` : armazena a frequência de cada símbolo;
- `PriorityQueue<HuffmanNode>` : seleciona automaticamente os dois nós de menor frequência durante a construção da árvore;
- `HuffmanNode` : representa cada nó da árvore de Huffman;
- `HashMap<String, String>` : representa o dicionário Huffman, associando cada símbolo ao seu código binário;
- `DecodeNode` : representa a árvore de decodificação usada na descompactação;
- `BitWriter` : agrupa os bits gerados em bytes reais antes de gravar no arquivo compactado;
- `CompressedFileHeader` : cabeçalho salvo no arquivo compactado, contendo o modo de compressão, o dicionário Huffman e a quantidade de bits válidos.

6. Formato do arquivo compactado

O arquivo compactado gerado pelo programa contém duas partes principais:

1. Cabeçalho serializado em Java, contendo:
 - modo de compressão usado: caractere ou palavra;
 - dicionário Huffman;
 - quantidade de bits válidos na área compactada.
2. Sequência de bytes compactados, formada pelos códigos Huffman agrupados em bytes.

O dicionário é salvo no próprio arquivo compactado. Assim, a descompactação não depende de reconstruir a árvore a partir das frequências, evitando problemas quando símbolos possuem a mesma frequência.

7. Processo de compactação

A compactação segue as seguintes etapas:

1. O programa recebe o arquivo de entrada e o arquivo de saída;
2. O arquivo é lido em UTF-8;
3. O programa divide o conteúdo em símbolos, dependendo do modo escolhido;
4. As frequências são contadas;
5. A árvore de Huffman é construída com uma fila de prioridade;
6. O dicionário é gerado percorrendo a árvore;
7. O arquivo é lido novamente e cada símbolo é substituído pelo código correspondente;
8. Os bits são agrupados de 8 em 8 para formar bytes;
9. O cabeçalho e os bytes compactados são gravados no arquivo final;
10. O programa exibe tamanho original, tamanho compactado, taxa de compressão, redução obtida e tempo de execução.

8. Processo de descompactação

A descompactação segue as seguintes etapas:

1. O programa lê o arquivo compactado;
2. O cabeçalho é recuperado;
3. O dicionário Huffman é utilizado para montar uma árvore de decodificação;
4. Os bytes compactados são lidos bit a bit;
5. A cada código completo encontrado, o símbolo original é escrito no arquivo de saída;
6. O processo continua até atingir a quantidade de bits válidos registrada no cabeçalho;
7. O arquivo reconstruído é salvo em UTF-8.

9. Mensuração do tempo de execução

O tempo de execução foi medido diretamente no programa utilizando `System.nanoTime()`.

A medição é iniciada imediatamente antes da operação de compactação ou descompactação e finalizada logo após a conclusão da operação. O resultado é convertido para milissegundos.

Fórmula usada no código:

```
tempo_ms = (tempo_final_ns - tempo_inicial_ns) / 1.000.000
```

10. Taxa de compressão

A taxa de compressão foi calculada dividindo o tamanho do arquivo compactado pelo tamanho do arquivo original.

```
taxa = tamanho_compactado / tamanho_original
```

A redução percentual foi calculada por:

```
redução = 1 - taxa
```

Exemplo:

```
Arquivo original: 100 MB
Arquivo compactado: 40 MB
Taxa: 40 / 100 = 0,40 = 40%
Redução: 1 - 0,40 = 0,60 = 60%
```

11. Resultados obtidos

Todos os arquivos descompactados foram testados por meio da função `fc.exe /b` do Windows. Nenhuma discrepância entre os arquivos descompactados e os originais foi detectada.

11.1 Arquivo de aproximadamente 10 MB

Codificação por caractere

- Tamanho original: *10.26 MB*
- Tamanho compactado: *6.73 MB*
- Taxa compactado/original: *65.60%*
- Redução obtida: *34.40%*
- Tempo de compactação: *768 ms*
- Tempo de descompactação: *327 ms*

Codificação por palavra

- Tamanho original: *10.26 MB*
- Tamanho compactado: *1.22 MB*
- Taxa compactado/original: *11.85%*
- Redução obtida: *88.15%*
- Tempo de compactação: *304 ms*
- Tempo de descompactação: *86 ms*

11.2 Arquivo de aproximadamente 50 MB

Codificação por caractere

- Tamanho original: *51.30 MB*
- Tamanho compactado: *33.65 MB*
- Taxa compactado/original: *65.59%*
- Redução obtida: *34.41%*
- Tempo de compactação: *2286 ms*
- Tempo de descompactação: *1454 ms*

Codificação por palavra

- Tamanho original: *51.30 MB*
- Tamanho compactado: *6.08 MB*
- Taxa compactado/original: *11.85%*
- Redução obtida: *88.15%*
- Tempo de compactação: *1202 ms*
- Tempo de descompactação: *357 ms*

11.3 Arquivo de aproximadamente 200 MB

Codificação por caractere

- Tamanho original: *200.00 MB*
- Tamanho compactado: *130.02 MB*
- Taxa compactado/original: *65.01%*
- Redução obtida: *34.99%*
- Tempo de compactação: *9812 ms*
- Tempo de descompactação: *4614 ms*

Codificação por palavra

- Tamanho original: *200.00 MB*
- Tamanho compactado: *24.31 MB*
- Taxa compactado/original: *12.15%*
- Redução obtida: *87.85%*
- Tempo de compactação: *4353 ms*
- Tempo de descompactação: *1341 ms*

12. Conclusão

Com base nos testes realizados, conclui-se que o programa atendeu ao objetivo proposto, permitindo compactar e descompactar arquivos-texto utilizando o algoritmo de Huffman nos modos por caractere e por palavra. A validação dos arquivos descompactados demonstrou que a compressão realizada é sem perda, pois os arquivos reconstruídos permaneceram idênticos aos originais.

A análise dos resultados mostrou que a codificação por palavra apresentou melhor taxa de compressão nos arquivos testados, ficando com tamanho final consideravelmente menor em comparação com a codificação por caractere. Isso ocorreu porque os arquivos possuíam muitas repetições de palavras e padrões textuais, o que favoreceu a criação de códigos menores para tokens frequentes.

Também foi observado que o tempo de execução aumentou conforme o tamanho dos arquivos, comportamento esperado em razão do maior volume de dados processados. Ainda assim, os tempos obtidos foram adequados para arquivos da ordem de megabytes, incluindo o teste com arquivo de aproximadamente 200 MB. Dessa forma, a implementação se mostrou funcional para o cenário solicitado no trabalho e permitiu comparar empiricamente as duas estratégias de codificação.